Scaling PostgreSQL to 800M ChatGPT Users

Source: Arpit Bhayani Dissection • OpenAI
Topic: Single Primary Monolith, 50 Replicas, PgBouncer

1. The Monolith Paradox: Why OpenAI Avoided Sharding

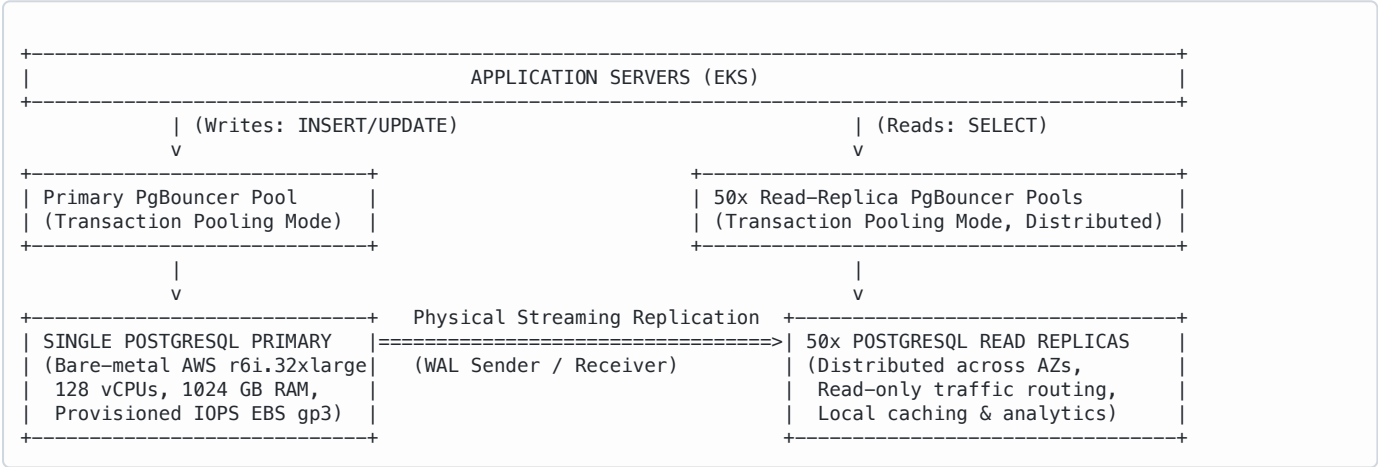
During ChatGPT's meteoric ascent from 0 to over 800 million active users, standard industry wisdom dictated sharding user metadata, conversation headers, and account graphs across hundreds of distributed SQL or NoSQL partitions. OpenAI engineers explicitly rejected this path. Premature sharding imposes catastrophic architectural tax: distributed two-phase commits (2PC), cross-shard joins, loss of foreign key integrity, re-sharding operational nightmares, and amplified failure domains.

Instead, OpenAI doubled down on a **single PostgreSQL primary writer** paired with over **50 read replicas**. By strictly analyzing the workload profile, OpenAI realized that ChatGPT's interaction model is overwhelmingly read-heavy (~95% reads vs 5% writes), allowing extreme vertical scaling of writes and massive horizontal pooling of reads.

> **Core Engineering Axiom:** The single primary architecture provides ACID guarantees, strict serializability without cross-node network consensus rounds, zero distributed transactions, and total operational simplicity during hypergrowth.

2. Topology & Read/Write Segregation Architecture

The cluster topology separates the write path from the read path through dedicated PgBouncer connection pooling tiers and smart application-level routing.



3. PgBouncer & Connection Lifecycle Management

PostgreSQL implements a process-per-connection model. Spawning thousands of direct client connections saturates process table memory, causes catastrophic OS context-switching, and triggers memory bloat. OpenAI places PgBouncer proxies immediately before the database instances:

- **Transaction Pooling Mode:** Client connections lease a server database connection only for the precise duration of a single transaction or query, releasing it immediately upon completion. 100,000+ app threads map cleanly to ~300 backend Postgres worker processes.
- **Eliminating Prepared Statement Overhead:** By tuning PgBouncer's `server_reset_query` and client connection life-cycles, stale memory allocations and connection leaks are eliminated.
- **Replication Lag Tolerant Reads:** Read queries that tolerate small replication drift (conversation titles, user settings) hit round-robin replica pools; strong-consistency reads bypass to the primary via read-your-writes session tokens.

Application-Layer Hardening & Write Path Optimization

4. Application-Layer Write Mitigation & Offloading

A single primary database can easily process 10,000 to 20,000 write IOPS if it is shielded from non-critical metadata updates. OpenAI instituted strict application-level guardrails to preserve primary IOPS:

Workload Type	Naive Implementation	OpenAI Production Architecture	Primary Impact
User Token Consumption	Synchronous `UPDATE users SET tokens = tokens - N`	Buffered in Redis counters; batch flushed via background worker every 60s	99.2% write QPS reduction on primary
Full Token Stream Telemetry	Raw prompt/response tokens written to PostgreSQL	Streamed directly to Blob Storage (S3/Azure Blob) + ClickHouse/Kafka	Zero WAL pollution from streaming LLM responses
Chat History Updates	Direct append on every message fragment	Appended to local state; persisted as single chunk upon turn completion	Consolidates hundreds of intermediate updates into 1 write
Session Heartbeats	`UPDATE sessions SET last_seen = NOW()`	Cached in Redis TTL; written to Postgres lazily with jitter	Completely offloaded from hot transactional path

5. Deep PostgreSQL Engine Tuning & WAL Optimization

To prevent Write-Ahead Log (WAL) write bottlenecks and disk queue spikes, PostgreSQL parameters were aggressively tuned for high-concurrency NVMe / provisioned IOPS storage:

- **max_connections:** Capped at 500 on the database engine itself to avoid internal lock contention (lwlock, procarray spinlocks), letting PgBouncer absorb the 50,000+ client bursts.
- **shared_buffers & OS Cache:** Sized to 25% of total system RAM (~256GB), reserving the remaining 768GB for Linux Page Cache to ensure hot indexes and tables sit permanently in memory.
- **wal_buffers:** Set to 64MB (well above default 16MB) to ensure large transaction bursts do not block on WAL flush syncs.
- **checkpoint_completion_target:** Tuned to `0.9` with large `max_wal_size = 64GB` to smooth out I/O writes across checkpoint intervals, eliminating severe periodic disk write freezes.
- **Partial & Covering Indexes:** B-Tree indexes use `INCLUDE` clauses to enable Index-Only Scans on read replicas, completely bypassing heap page fetches for conversational metadata.

6. Replication Lag Management: Streaming Physical WAL

With 50+ read replicas, replica lag can induce read-after-write inconsistencies. OpenAI relies on physical streaming replication:

> **Physical WAL Streaming vs Logical Replication:** Physical WAL replicates byte-for-byte binary block modifications rather than SQL statements or row changes. This runs with orders-of-magnitude lower CPU overhead, sub-millisecond propagation latency across the local availability zone, and near-instant recovery.

Failure Modes, SEVs & Practical Engineering Principles

Source: Arpit Bhayani Dissection • OpenAI
Focus: SEVs, Thundering Herds, Failover & Takeaways

7. Real-World Failure Modes & Severe Incidents (SEVs)

Running at this extreme scale exposed several critical failure modes that OpenAI engineers had to systematically neutralize:

A. Cache Invalidation Stampedes (Thundering Herd)

When hot cached keys (e.g., global system announcements, model capability flags, popular shared prompts) expired, thousands of simultaneous application workers bypassed Redis and hammered the PostgreSQL replicas with identical expensive queries.

> **The Fix: Singleflight & Probabilistic Early Expiration:** Implemented singleflight request coalescing (collapsing N identical concurrent fetches into a single backend query) and XFetch / early refreshing before TTL expiration to ensure caches never empty completely.

B. Connection Spike Starvation

During cold starts or traffic bursts, hundreds of ECS/EKS pods spun up simultaneously, creating connection storm surges that exhausted PgBouncer connection pools.

> **The Fix: Exponential Backoff with Decorrelated Jitter:** Application retry policies were re-architected with circuit breakers and full jitter: $t_{wait} = \min(t_{max}, ext{random}(0, 2^{ ext{retry}} imes t_{base}))$. Aggressive request shedding dropped lower-priority background jobs.

8. Automated Failover & High Availability Topology

A single primary means that if the primary dies, writes halt. OpenAI uses **Patroni with Raft/etcd** to provide automated, split-brain-safe primary failover:

- **Health Heartbeats:** Patroni nodes constantly publish heartbeats to a distributed etcd consensus cluster with a strict 10-second TTL.
- **Leader Election:** If the primary fails to renew its lease, etcd triggers an automated leader election among the designated synchronous standby replicas.
- **Zero Data Loss (RPO = 0):** At least one standby replica operates in synchronous streaming replication mode (`synchronous_commit = on`), guaranteeing zero committed transaction loss during failover within < 20 seconds.

9. First-Principles System Design Takeaways

Decision Vector	Common Industry Misconception	OpenAI Production Reality
Scaling Strategy	Shard early to prepare for massive scale	Exhaust vertical scaling + read replication first; avoid distributed complexity
Storage Model	Store all app data in primary database	Keep only strict transactional state in Postgres; offload blobs to S3 & analytics to OLAP
Connection Pool	Direct app connections with large pool sizes	Strict PgBouncer transaction pooling; keep database backend process count below 500
Write Path	Write synchronously on every client mutation	Buffer, batch, and jitter writes at the application/Redis tier before touching Postgres